Stručni kurs Razvoj bezbednog softvera

Izveštaj

Pronađene ranjivosti u projektu “RealBookStore”

Luka Arambašić 169/2020 Matematički fakultet

Sadržaj

Uvod ... 3
O veb aplikaciji ... 3
Kratak pregled rezltata testiranja ... 3
SQL Injection ... 4
Napad: Ubacivanje novog korisnika u tabelu “persons” (SQL Injection) preko polja za komentar ... 4
Odbrana: korišćenje parametara umesto direktne konkatenacije ... 5
SQL Injection + XSS (Cross-site scripting) ... 6
Napad: Ubacivanje maliciozne skripte (XSS) u jedan od atributa novoubačenog korisnika koji se ubacuje preko SQL Injection-a kroz polje za pretragu osoba ... 6
Odbrana: Za SQL injection identično, otklanjamo direktnu konkatenaciju, a za XSS menjamo .innerHtml sa .textContent ... 9
Cross-site request forgery ... 10
Napad: Navodimo žrtvu da klikne link koji šalje zahtev koji menja podatke ... 10
Odbrana: Implementiramo logiku tokena ... 11

Uvod

Ovaj izveštaj se bavi ranjivostima u dole opisanoj veb aplikaciji.

O veb aplikaciji

RealBookStore je veb aplikacija koja pruža mogućnosti pretrage, ocenjivanje i komentarisanja knjiga.

Aplikacija RealBookStore omogućava sledeće:

- Pregled i pretragu knjiga.
- Dodavanje nove knjige.
- Detaljan pregled knjige kao i komentarisanje i ocenjivanje knjige.
- Pregled korisnika aplikacije.
- Detaljan pregled podataka korisnika

Kratak pregled rezltata testiranja

Na projektu je korišćen alat SonarQube za statičku analizu koda i pronalaženje ranjivosti.

Pronađeno je 20 ranjivosti.

19 ranjivosti su SQL injection od kojih je samo jedna lažna pozitivna.

1 ranjivost je CSRF

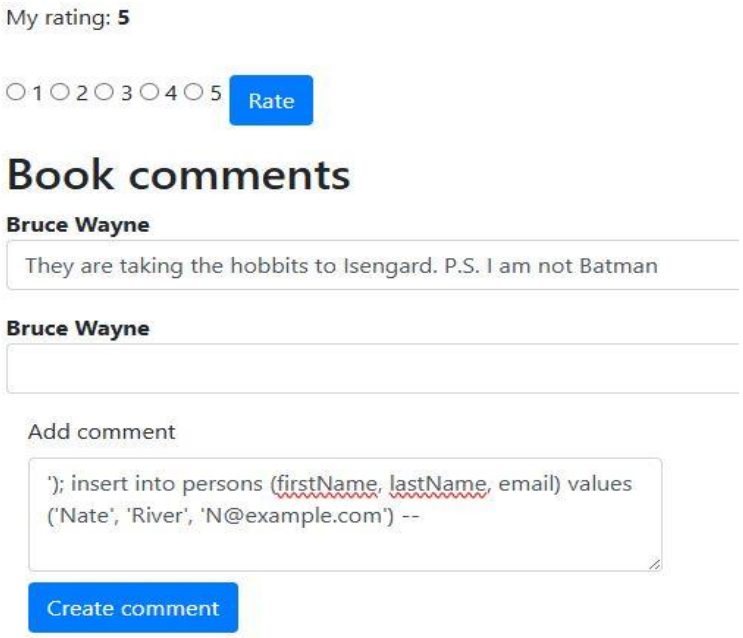
Alat je svim ranjivostima dodelio “High” nivo opasnosti.

SQL Injection

Napad: Ubacivanje novog korisnika u tabelu “persons” (SQL Injection) preko polja za komentar

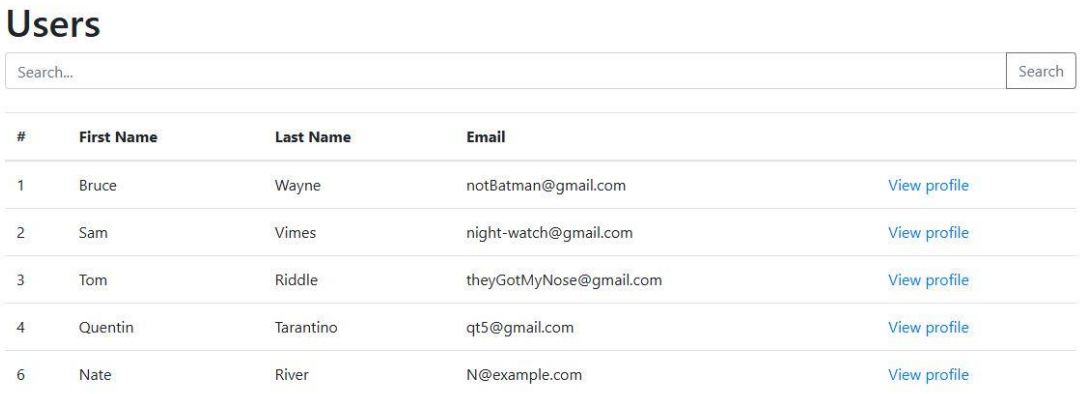
Na stranici za unošenje komentara uneti sledeći kod u polje za komentar:

'); INSERT INTO persons (firstName, lastName, email) VALUES ('Nate', 'River', N@example.com'); --



Kod unosi proizvoljnog korisnika u tabelu “persons”.

Nakon toga se na stranici “persons” može uočiti dodat novi korisnik:

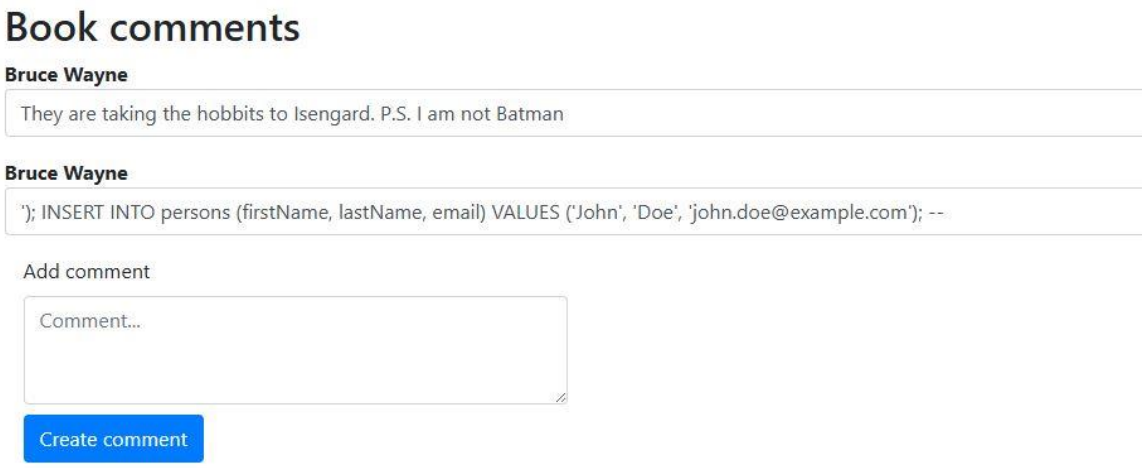


#	First Name	Last Name	Email	
1	Bruce	Wayne	notBatman@gmail.com	View profile
2	Sam	Vimes	night-watch@gmail.com	View profile
3	Tom	Riddle	theyGotMyNose@gmail.com	View profile
4	Quentin	Tarantino	qt5@gmail.com	View profile
6	Nate	River	N@example.com	View profile

Odbrana: korišćenje parametara umesto direktne konkatenacije

```
public void create(Comment comment) {  1 usage  Dragojevic, Uros *
    // String query = "insert into comments(bookId, userId, comment) values (" + comment
    String query = "INSERT INTO comments (bookId, userId, comment) VALUES (?, ?, ?)";


    try (Connection connection = dataSource.getConnection();
         //Statement statement = connection.createStatement();
         PreparedStatement statement = connection.prepareStatement(query)
    ) {
        statement.setInt( parameterIndex: 1, comment.getBookId());
        statement.setInt( parameterIndex: 2, comment.getUserId());
        statement.setString( parameterIndex: 3, comment.getComment());

        int rowsAffected = statement.executeUpdate();

        if (rowsAffected > 0) {
            System.out.println("Comment inserted successfully.");
        }
        statement.execute(query);

    } catch (SQLException e) {
```

Nakon izmenjenog koda i ponovnog pokušaja napada možemo uočiti da ne dolazi do neželjenog ponašanja naše aplikacije, već komentar ostaje u formi u kojoj je unet

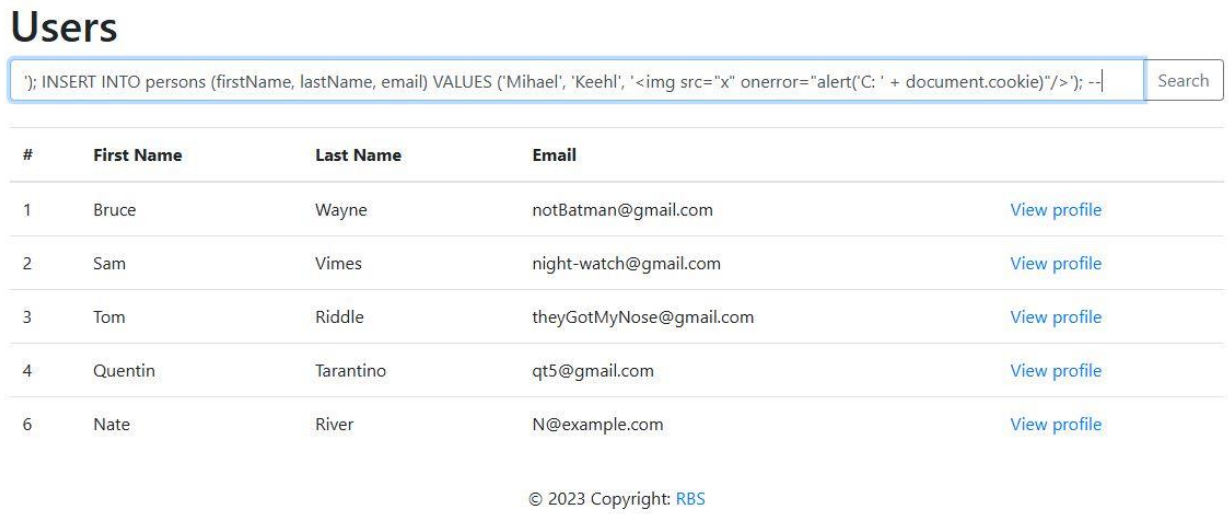


SQL Injection + XSS (Cross-site scripting)

Napad: Ubacivanje maliciozne skripte (XSS) u jedan od atributa novoubačenog korisnika koji se ubacuje preko SQL Injection-a kroz polje za pretragu osoba

U polje za pretragu se unosi sličan kod kao za prošli napad, sa dodatkom da ćemo umesto email adrese kao atribut novog korisnika dodati kod za skriptu koja će u konzoli razvojnog alata ispisati cookie ID:

```
'); INSERT INTO persons (firstName, lastName, email) VALUES ('Mihael', 'Keehl',
        '<img src="x" onerror="console.log('Stolen Cookie:', document.cookie)"/>'); --
```



Pritiskom na dugme search bi trebalo da se korisnik ubaci u bazu i ponovo da se aktivira pretraga kako bi se aktivirala skripta. Ali nailazi se na problem. Ne dozvoljava se ubacivanje takvog koda u korisničko polje. To može biti iz raznih razloga jer posoje odbrane na više nivoa od XSS napada koje su veoma česta pojava. Greška koja izlazi u konzoli:

Ali ne treba odustajati! Primetimo da se ova interakcija može izvršiti u dva koraka s obzirom da imamo mogućnost da kliknemo na dugme “View profile” i da naknadno

izmenimo vrednosti atributa za željenog korisnika. Korisnika ćemo uneti preko SQL Injection-a a naknadno ćemo u polje za email ubaciti našu skriptu:

Users

'); INSERT INTO persons (firstName, lastName, email) VALUES ('Mihael', 'Keehl', 'SCRIPT HERE'); -- Search

You searched for '); INSERT INTO persons (firstName, lastName, email) VALUES ('Mihael', 'Keehl', 'SCRIPT HERE'); --

#	First Name	Last Name	Email	
1	Bruce	Wayne	notBatman@gmail.com	View profile
2	Sam	Vimes	night-watch@gmail.com	View profile
3	Tom	Riddle	theyGotMyNose@gmail.com	View profile
4	Quentin	Tarantino	qt5@gmail.com	View profile
6	Nate	River	N@example.com	View profile

© 2023 Copyright: RBS

Korisnik je sada dodat.

Users

Search... Search

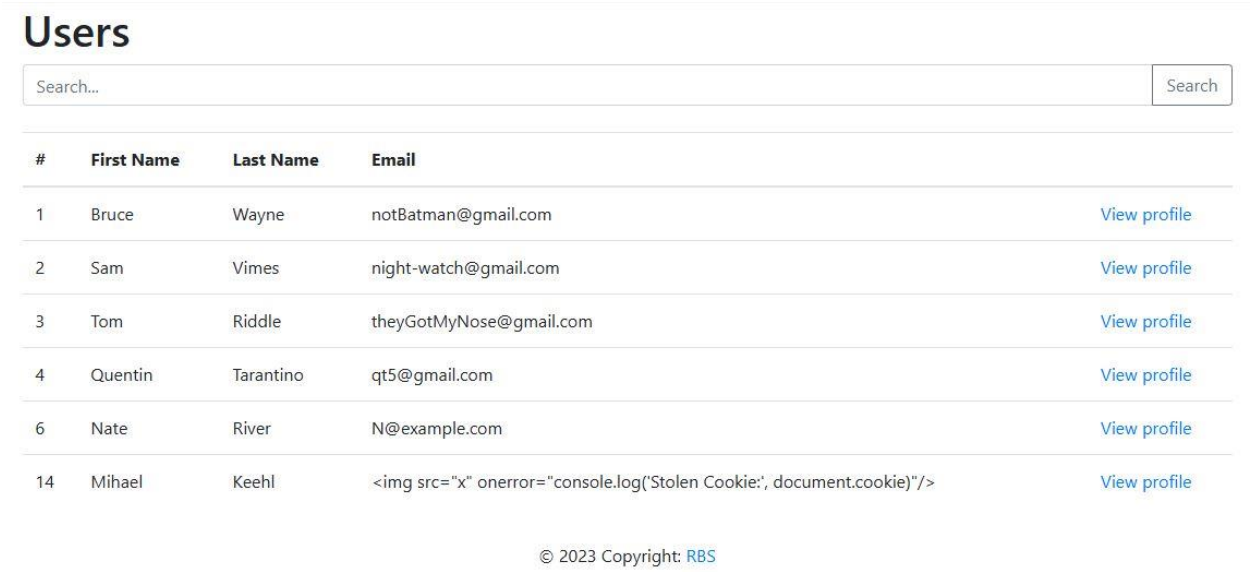
#	First Name	Last Name	Email	
1	Bruce	Wayne	notBatman@gmail.com	View profile
2	Sam	Vimes	night-watch@gmail.com	View profile
3	Tom	Riddle	theyGotMyNose@gmail.com	View profile
4	Quentin	Tarantino	qt5@gmail.com	View profile
6	Nate	River	N@example.com	View profile
12	Mihael	Keehl	SCRIPT HERE	View profile

© 2023 Copyright: RBS

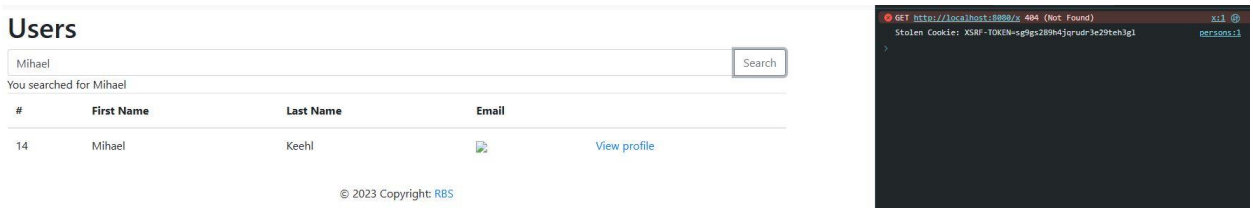
Prelazimo na očekivano menjanje podataka:

U skripti tražimo nepostojeću sliku “x” kako bismo isforsirali grešku.

Podaci su uspešno promenjeni.



Ostaje samo da aktivira skripta klikom na dugme “Search” i dobijamo Cookie ID.



Odbrana: Za SQL injection identično, otklanjamo direktnu konkatenaciju, a za XSS menjamo .innerHtml sa .textContent

```java
public List<Person> search(String searchTerm) throws SQLException {  1 usage  Dragojevic, Uros *
    List<Person> personList = new ArrayList<>();
    /*String query = "SELECT id, firstName, lastName, email FROM persons WHERE UPPER(firstN
            " OR UPPER(lastName) like UPPER('%" + searchTerm + "%')";

     */
    String query = "SELECT id, firstName, lastName, email FROM persons " +
            "WHERE UPPER(firstName) LIKE ? OR UPPER(lastName) LIKE ?";

    String searchPattern = "%" + searchTerm.toUpperCase() + "%";

    try (Connection connection = dataSource.getConnection();
         PreparedStatement statement = connection.prepareStatement(query)) {

        statement.setString( parameterIndex: 1, searchPattern);
        statement.setString( parameterIndex: 2, searchPattern);

        try (ResultSet rs = statement.executeQuery()) {
            while (rs.next()) {
                personList.add(createPersonFromResultSet(rs));
            }
        }
    }
    return personList;
}
```

```html
<script>
    window.addEventListener('load', function() {
        let search = function () {  Show usages  Dragojevic, Uros *
            const searchTerm = document.getElementById("searchInput").value;

            fetch('/persons/search?searchTerm=' + searchTerm)
                .then(function (result) {return result.json()})
                .then(function(persons) {
                    const tableContent = document.getElementById("tableContent");
                    tableContent.innerHTML = '';

                    persons.forEach(function(person) {
                        const tableRowElement = document.createElement("tr");
                        let tdElement = document.createElement("td");
                        //tdElement.innerHTML = person.id;
                        tdElement.textContent = person.id;
                        tableRowElement.appendChild(tdElement);
                        tdElement = document.createElement("td");
                        //tdElement.innerHTML = person.firstName;
                        tdElement.textContent = person.firstName;
                        tableRowElement.appendChild(tdElement);
                        tdElement = document.createElement("td");
                        //tdElement.innerHTML = person.lastName;
                        tdElement.textContent = person.lastName;
                        tableRowElement.appendChild(tdElement);
                        tdElement = document.createElement("td");
                        //tdElement.innerHTML = person.email;
                        tdElement.textContent = person.email;
                        tableRowElement.appendChild(tdElement);
                        tdElement = document.createElement("td");
                        tdElement.innerHTML = '<a href="/persons/' + person.id + '">V
                        tableRowElement.appendChild(tdElement);

                        tableContent.appendChild(tableRowElement);
```

Nakon ovih promena, isprobavamo ponovo napad i može se uočiti da su ove ranjivosti otklonjene.

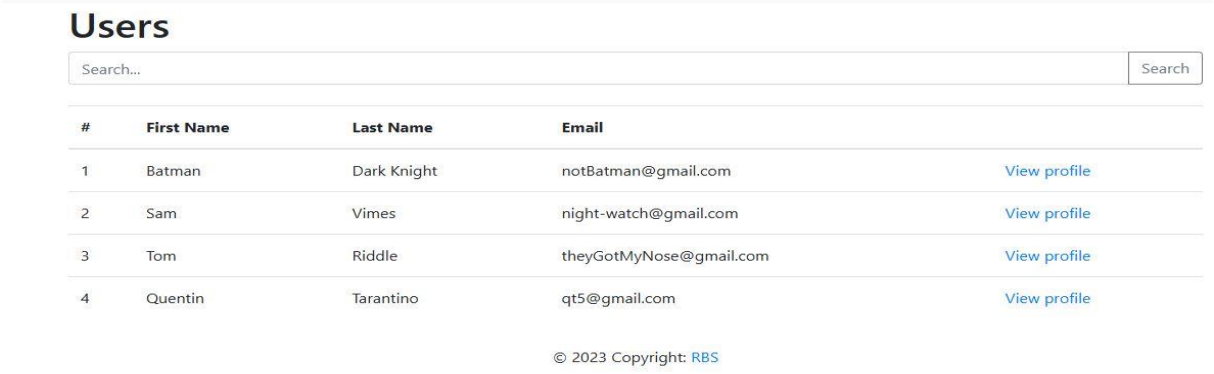
Cross-site request forgery

Napad: Navodimo žrtvu da klikne link koji šalje zahtev koji menja podatke

Zahtev koji šaljemo:

```
<script>
    function exploit() {  Show usages  Dragojevic, Uros *
        // Scripted CSRF Request
            const formData = new FormData();
            formData.append('id', 1);
            formData.append('firstName', 'Batman');
            formData.append('lastName', 'Dark Knight')
            fetch('http://localhost:8080/update-person',
                {method: 'POST', body: formData, credentials: 'include'});
    }
</script>
```

Žrtva klikne na našu stranicu i kada ode na stranicu ”persons” podaci budu promenjeni:



Odbrana: Implementiramo logiku tokena

```
@GetMapping("/persons/{id}")  Dragojevic, Uros *
public String person(@PathVariable int id, Model model, HttpSession session) {
    String csrf = session.getAttribute( s: "CSRF_TOKEN").toString();
    model.addAttribute( attributeName: "CSRF_TOKEN", session.getAttribute( s: "CSRF_TOKEN"));
    model.addAttribute( attributeName: "person", personRepository.get("" + id));
    return "person";
}
```

```
    <input type="hidden" name="id" class="form-control" id="id" th:value="${person.id}">
    <input type="hidden" name="csrfToken" th:value="${CSRF_TOKEN}">
    <button type="submit" class="btn btn-primary">Save</button>
```

```
@PostMapping("/update-person")  Dragojevic, Uros *
public String updatePerson(Person person, HttpSession session, @RequestParam("csrfToken") String csrfToken) throws AccessDeniedException {
    String csrf = session.getAttribute( s: "CSRF_TOKEN").toString();
    if (!csrf.equals(csrfToken)) {
        throw new AccessDeniedException("Forbidden");
    }
    personRepository.update(person);
    return "redirect:/persons/" + person.getId();
}
```

I sada nakon što pokušamo napad, ništa se ne dešava

Users

Search... Search

#	First Name	Last Name	Email	
1	Bruce	Wayne	notBatman@gmail.com	View profile
2	Sam	Vimes	night-watch@gmail.com	View profile
3	Tom	Riddle	theyGotMyNose@gmail.com	View profile
4	Quentin	Tarantino	qt5@gmail.com	View profile

Implementacija autorizacije

Rađeno pomoću Thymeleaf-a i Spring security framework-a.

Permisije i role:

Permisija	ADMIN	MANAGER	REVIEWER
ADD_COMMENT	*	*	*
VIEW_BOOKS_LIST	*	*	*
CREATE_BOOK	*	*	X
VIEW_PERSONS_LIST	*	*	X
VIEW_PERSON	*	X	X
UPDATE_PERSON	*	* - pogledaj napomenu	* - pogledaj napomenu
VIEW_MY_PROFILE	*	*	*
RATE_BOOK	*	*	*

Kratko objašnjenje permisija:

Permisija	Opis
ADD_COMMENT	Dozvoljava korisniku da doda komentar.
VIEW_BOOKS_LIST	Dozvoljava korisniku da vidi i pretraži listu knjiga.
CREATE_BOOK	Dozvoljava korisniku da unese novu knjigu.
VIEW_PERSONS_LIST	Dozvoljava korisniku da vidi i pretraži listu korisnika.
VIEW_PERSON	Dozvoljava korisniku da vidi detalje osobe.
UPDATE_PERSON	Dozvoljava korisniku da promeni detalje osobe (isto važi za brisanje korisnika).
VIEW_MY_PROFILE	Dozvoljava pregled sopstvenog profila.
RATE_BOOK	Dozvoljava da korisnik oceni knjigu.

Prvo ažuriramo bazu mapiranjem permisija i rola:

```sql
insert into user_to_roles(userId, roleId)
values ( userId 1,  roleId 3),
       ( userId 2,  roleId 3),
       ( userId 3,  roleId 1),
       ( userId 4,  roleId 2);

insert into permissions(name)
values ( name 'ADD_COMMENT'),
       ( name 'VIEW_BOOKS_LIST'),
       ( name 'CREATE_BOOK'),
       ( name 'VIEW_PERSONS_LIST'),
       ( name 'VIEW_PERSON'),
       ( name 'UPDATE_PERSON'),
       ( name 'VIEW_MY_PROFILE'),
       ( name 'RATE_BOOK')
       ;

insert into role_to_permissions(roleId, permissionId)
values ( roleId 1, permissionId 1),
       ( roleId 1, permissionId 2),
       ( roleId 1, permissionId 3),
       ( roleId 1, permissionId 4),
       ( roleId 1, permissionId 5),
       ( roleId 1, permissionId 6),
       ( roleId 1, permissionId 7),
       ( roleId 1, permissionId 8),
       ( roleId 2, permissionId 1),
       ( roleId 2, permissionId 2),
       ( roleId 2, permissionId 3),
       ( roleId 2, permissionId 4),
       ( roleId 2, permissionId 6),
       ( roleId 2, permissionId 7),
       ( roleId 2, permissionId 8),
       ( roleId 3, permissionId 1),
       ( roleId 3, permissionId 2),
       ( roleId 3, permissionId 6),
       ( roleId 3, permissionId 7),
       ( roleId 3, permissionId 8),
```

Potrebno je za svaku permisiju proveriti i frontend i backend (ukoliko postoji).

Za backend se dodaje anotacija:

```
@PreAuthorize("hasAuthority('NAZIV_PERMISIJE')")
```

Za frontend se html elementu dodaje atribut:

```
sec:authorize="hasAuthority('NAZIV_PERMISIJE')"
```

U kodu:

```
<button id="createComment" class="btn btn-primary mt-2" type="submit" sec:authorize="hasAuthority('ADD_COMMENT')" >Create comment</button>
```

```
    @PostMapping(value = "/comments")   Dragojevic, Uros
    @PreAuthorize("hasAuthority('ADD_COMMENT')")
    public String createComment(@ModelAttribute Comment c
        User user = (User) authentication.getPrincipal();
        comment.setUserId(user.getId());
        commentRepository.create(comment);

        return "redirect:/books/" + comment.getBookId();
    }
}
```

```
    @GetMapping(value = "/books/search", produces =
    @ResponseBody
    @PreAuthorize("hasAuthority('VIEW_BOOK_LIST')")
    public List<Book> search(@RequestParam("query") St
        return bookRepository.search(query);
    }

    @PostMapping("/books")   Dragojevic, Uros
    @PreAuthorize("hasAuthority('CREATE_BOOK')")
    public String createBook(NewBook book) {
        List<Genre> genreList = this.genreRepository.g
        List<Genre> genresToInsert = book.getGenres().
        long id = bookRepository.create(book, genresTo
        return "redirect:/books?id=" + id;
    }
```

```
<a href="/create-form" sec:authorize="hasAuthority('CREATE_BOOK')">Add book</a>
```

```
<table class="table" sec:authorize="hasAuthority('VIEW_BOOKS_LIST')">
```

```
<table class="table" sec:authorize="hasAuthority('VIEW_PERSONS_LIST')">
    <thead>
```

```
    @GetMapping(value = "/persons/search", produces =
    @ResponseBody
    @PreAuthorize("hasAuthority('VIEW_PERSONS_LIST')")
    public List<Person> searchPersons(@RequestParam Strin
        return personRepository.search(searchTerm);
    }
}
```

```
@PostMapping("/update-person")   LukaArambasic01 +1
@PreAuthorize("hasAuthority('UPDATE_PERSON')")
public String updatePerson(Person person, HttpSession session, @Re
    String csrf = session.getAttribute( s: "CSRF_TOKEN").toString()
    if (!csrf.equals(csrfToken)) {
```

```
<button type="submit" class="btn btn-primary" sec:authorize="hasAuthority('UPDATE_PERSON')">Save</button>
```

```
@GetMapping("/myprofile")   Dragojevic, Uros
@PreAuthorize("hasAuthority('VIEW_MY_PROFILE')")
public String self(Model model, Authentication authentication) {
    User user = (User) authentication.getPrincipal();
    model.addAttribute( attributeName: "person", personRepository.get(
```

```
<a class="nav-link" th:href="@{/myprofile}" sec:authorize="hasAuthority('VIEW_MY_PROFILE')">My Profile</a>
```

```
    @PostMapping(value = "/ratings")   Dragojevic, Uros
    @PreAuthorize("hasAuthority('RATE_BOOK')")
    public String createOrUpdateRating(@ModelAttribute R
        rating.setUserId(1);
```

```
<button id="rateBook" class="btn btn-primary mt-2" type="submit" sec:authorize="hasAuthority('RATE_BOOK')">Rate</button>
```